

Manual de netlab

Flujo de puesta en marcha y funcion de cada modulo

Que hace, para que sirve y que problema resuelve

Generado el 5/10/2026

Contenido

Informe generado automaticamente por netlab

Como leer este manual

Cada modulo se explica con las mismas cuatro preguntas: que hace, para que sirve, que problema resuelve y como se recorre. Las dos primeras describen el comportamiento; la tercera es la que importa cuando alguien pregunta para que sirve una pieza concreta; la cuarta es el recorrido real, paso a paso.

Modulos

Autenticacion y sesiones - Quien entra, con que prueba y durante cuanto tiempo.

Configuracion del administrador principal - El primer ingreso al sistema obliga a crear la cuenta que gobierna todas las demas.

Administracion de usuarios - El administrador principal da de alta, edita y desactiva el resto de cuentas.

Auditoria - Registro de quien hizo cada accion y desde que IP.

Herramientas de diagnostico - El catalogo de comprobaciones que netlab puede ejecutar.

Historial - Todas las ejecuciones, con quien las lanzo y que parametro se uso.

Informes y enlaces compartidos - Como se entrega el resultado y como se comparte con alguien de fuera.

Metricas - Contadores y tiempos de las peticiones, para vigilar el comportamiento del servicio.

Apendices

Las herramientas de diagnostico, con su identificador y su proposito.

La revision de preparaci3n para produccion, con el estado de cada comprobacion.

El despliegue en la VPS, con los pasos y el orden correcto.

Resumen

Informe generado automaticamente por netlab

Que es netlab

Suite de diagnostico de red, DNS, IP y correo

netlab es una aplicacion web de diagnostico de infraestructura. Reune en una sola interfaz las comprobaciones que antes eran scripts sueltos: analisis de IP y de reputacion, consultas DNS, verificacion de correo, validador de SMTP, reparto de subredes y comprobacion de sitios web. Cada ejecucion produce un informe con veredicto, resumen, detalle, hallazgos y recomendacion, descargable en cinco formatos.

Arquitectura

Como encajan las piezas

Servidor Node.js con Express en src/server, que expone una API JSON y sirve la interfaz compilada.

SPA en React y Vite en web/src, compilada a web/dist.

Herramientas de diagnostico en src/tools, descubiertas al arrancar.

Capa comun en src/core: configuracion, errores del dominio, resultado normalizado, enmascarado de secretos y utilidades de red.

Formatos de salida en src/formats, con PDF, HTML, JSON, Markdown y TXT a partir del mismo resultado.

Persistencia en SQLite: usuarios, historial, enlaces compartidos y auditoria.

Flujo de arranque

Del primer ingreso al diagnostico

1. Arranca el servidor. Carga la configuracion, descubre las herramientas y abre el puerto.
2. Se comprueba el estado de la autentificacion. Si esta activa y no hay ningun usuario, el sistema arranca en modo setup.
3. La pagina consulta el estado del setup. Si falta el administrador, muestra el formulario de creacion en lugar del login.
4. Se crea el administrador principal. A partir de ese momento, el setup queda cerrado y ya no se puede repetir.
5. El administrador entra con su cuenta y usa las herramientas. Solo el rol admin accede a la gestion de usuarios y a la auditoria.
6. Cada ejecucion se valida, se limita, se enmascara antes de guardarse y queda en el historial de quien la lanzo.
7. El resultado se puede descargar en PDF, HTML, JSON, Markdown o TXT, y compartir mediante un enlace publico temporal.

Modulos

Informe generado automaticamente por netlab

Autenticacion y sesiones

Quien entra, con que prueba y durante cuanto tiempo.

Que hace

- Valida usuario y contrasena contra la tabla de usuarios de SQLite.
- Emite una cookie de sesion firmada, opaca y aleatoria, marcada HttpOnly y SameSite=Strict.
- Emite un token CSRF independiente que debe viajar en la cabecera X-CSRF-Token en toda peticion que modifique datos.
- Caduca la sesion a las 8 horas y el token CSRF a las 2 horas.
- Limita a 8 los intentos de acceso por IP cada 15 minutos, con espera creciente.
- Cierra las sesiones al reiniciar el servidor: la sesion vive en memoria, no en disco.

Para que sirve

Para que solo quien tiene cuenta pueda lanzar diagnosticos desde el servidor, y para que una pagina de terceros no pueda suplantar al navegador que ya esta dentro.

Que soluciona

- Un puerto de diagnostico abierto a Internet es un escaner de ports gratuito para cualquiera que llegue al puerto.
- El robo de cookie por JavaScript de terceros: la cookie es HttpOnly, asi que no se puede leer desde la pagina.
- El CSRF: un sitio ajeno puede pedir al navegador que envíe la cookie, pero no puede conocer el token que va en la cabecera.
- La fuerza bruta: el limite por IP corta los intentos repetidos aunque las credenciales sean adivinables.

Flujo

1. El usuario envia POST /api/sesion con usuario y contrasena.
2. El servidor comprueba el limite de intentos de esa IP. Si se pasa, responde 429 con los segundos que faltan.
3. Se verifica la contrasena con script. Si falla, la respuesta es la misma para usuario inexistente y contrasena incorrecta.
4. Si es correcta, se crea el token de sesion y el token CSRF, se devuelve la cookie y el cliente guarda el CSRF.
5. Cada peticion posterior se identifica con la cookie; las que modifican datos ademas exigen la cabecera del CSRF.
6. DELETE /api/sesion cierra la sesion y borra la cookie.

Configuracion del administrador principal

El primer ingreso al sistema obliga a crear la cuenta que gobierna todas las demas.

Que hace

- Al cargar la pagina, el cliente consulta GET /api/setup/status.
- Si la tabla de usuarios esta vacia, el servidor responde que el setup esta pendiente y la web muestra el formulario de administrador principal en lugar del login.

- POST /api/setup/first-admin crea el primer usuario con rol admin. La contraseña se guarda con bcrypt, nunca en texto plano.
- En cuanto existe un usuario, la ruta responde 403 y deja de ser utilizable: el setup no se puede repetir.
- Toda la creacion queda registrada en la auditoria con el tipo setup_admin_creado y la IP de origen.
- Si la autentificacion esta desactivada (AUTH_ENABLED=false), el setup no aplica: el sistema ya corre como administrador local.

Para que sirve

Para que nunca exista un despliegue en produccion sin un administrador, y para que ese administrador sea siempre el primero que entro, no alguien que se registro antes que el.

Que soluciona

- El despliegue que se queda sin admin: hasta ahora habia que crear el admin con un comando manual por SSH, y ese paso se puede saltar.
- La ventana de tiempo en la que el sistema es accesible sin control de acceso.
- La duda de quien decide los permisos: el administrador principal queda registrado en auditoria desde el primer momento.

Flujo

1. Se abre la pagina y el navegador pide GET /api/setup/status.
2. Si la respuesta es completado: false, se muestra el formulario de configuracion del administrador principal.
3. Elige nombre de usuario y contraseña de al menos 8 caracteres, y la repite para confirmar.
4. El navegador envia POST /api/setup/first-admin.
5. El servidor comprueba que no haya ningun usuario. Si los hay, responde 403 y no hace nada.
6. Si no hay ninguno, crea el admin, lo registra en auditoria y responde 201.
7. La pagina vuelve a consultar el estado, ahora completado, y muestra el login normal.

Administracion de usuarios

El administrador principal da de alta, edita y desactiva el resto de cuentas.

Que hace

- GET /api/usuarios devuelve la lista de cuentas con rol, estado, fecha de creacion y ultimo acceso.
- POST /api/usuarios crea una cuenta nueva con rol user o admin. La contraseña minima es de 8 caracteres.
- PATCH /api/usuarios/:id cambia contraseña, rol o estado activo. El nombre de usuario no se puede cambiar.
- DELETE /api/usuarios/:id borra la cuenta, con la excepcion de que un administrador no puede borrarse a si mismo.
- Todas las operaciones exigen rol admin y token CSRF, y todas dejan rastro en la auditoria.
- El rol decide el alcance: un usuario normal usa las herramientas y ve su historial; un administrador accede tambien a esta pantalla y a la auditoria.

Para que sirve

Para dar acceso a un equipo sin compartir la misma contraseña, y para poder cortar el acceso de una persona concreta sin tocar el servidor.

Que soluciona

- La contraseña unica compartida por todo el equipo, que no permite saber quien hizo cada consulta.
- El caso de una persona que deja la empresa y sigue entrando porque su cuenta nunca se desactivo.
- La necesidad de saber que accounts existen y cuando se usaron por ultima vez.

Flujo

1. El administrador entra con su cuenta y abre /admin. Un usuario normal que intente esa ruta es devuelto a la pagina principal.
2. Pulsa Nuevo usuario y rellena nombre, contraseña, rol y estado.
3. El navegador envia POST /api/usuarios con el token CSRF.
4. El servidor exige rol admin, comprueba la longitud de la contraseña y que el nombre no este ocupado.
5. La cuenta se crea y el evento queda en la auditoria.
6. Para cambiar o desactivar una cuenta, pulsa Editar, ajusta lo necesario y guarda con PATCH.
7. Para eliminar una cuenta, pulsa Borrar y confirma. El propio administrador no aparece con boton de borrar.

Auditoria

Registro de quien hizo cada accion y desde que IP.

Que hace

- Registra accesos correctos e incorrectos, cierre de sesion, ejecuciones lanzadas y completadas, ejecuciones fallidas, altas y cambios de usuarios, borrados, descargas de informe y creacion de enlaces compartidos.
- GET /api/auditoria filtra por tipo de evento, por usuario y por antiguedad, con limite de registros.
- POST /api/auditoria/limpiar borra los registros anteriores a un numero de dias, 90 por defecto.
- Los datos sensibles se guardan ya enmascarados: las contraseñas nunca llegan al registro de auditoria.
- Solo accesible con rol admin.

Para que sirve

Para responder a la pregunta de quien lanzo una comprobacion concreta y que se obtuvo, y para detectar accesos que no corresponden a nadie del equipo.

Que soluciona

- La investigacion sin rastro: cuando un cliente pregunta por un informe de hace tres meses, no hay forma de saber si se llego a generar.
- El acceso indebido silencioso: una cuenta robada deja rastro con IP y hora.
- El crecimiento indefinido de la base de datos, con la limpieza por antiguedad.

Flujo

1. Cualquier accion relevante dispara un registro con tipo, usuario, IP y momento.
2. El administrador abre la auditoria y filtra por usuario o por tipo de evento.
3. Revisa la lista y, si hace falta, borra los registros antiguos con el endpoint de limpieza.

Herramientas de diagnostico

El catalogo de comprobaciones que netlab puede ejecutar.

Que hace

- Cada herramienta declara su identificador, su titulo, su descripcion y los campos que necesita.
- El servidor no tiene una lista fija: descubre los ficheros de src/tools al arrancar, de modo que anadir una comprobacion es dejar el archivo.
- El formulario se construye a partir de los campos declarados, sin JavaScript especifico por herramienta.
- Cada ejecucion se valida en el servidor antes de salir a Internet, y se enmascara antes de guardarse.
- El resultado se devuelve con un veredicto, un resumen, secciones de detalle, hallazgos con su recomendacion y la telemetria de la ejecucion.

Para que sirve

Para responder preguntas concretas de infraestructura y correo desde una sola herramienta, sin scripts sueltos por consola ni acceso al servidor.

Que soluciona

- Cada diagnostico era antes un script independiente con su propia paleta, su formato de informe y su forma de numerar paginas.
- Los errores no se podian distinguir de un dato malo: ahora cada fallo tiene su codigo, su mensaje y lo que hay que hacer.
- Los informes no eran reproducibles: se vuelve a renderizar desde el resultado guardado, no desde el PDF ya hecho.

Flujo

1. El usuario elige una herramienta del menu lateral.
2. Rellena el formulario, que se ha construido con los campos que declara la herramienta.
3. Pulsa Ejecutar y el navegador envia POST /api/run con el identificador y los parametros.
4. El servidor valida los parametros, comprueba el limite por minuto y por usuario, y lanza la comprobacion.
5. El resultado se guarda en el historial, ya enmascarado, y se devuelve al navegador.
6. El informe muestra veredicto, resumen, detalle, hallazgos y telemetria, y se puede descargar en PDF, HTML, JSON, Markdown o TXT.

Historial

Todas las ejecuciones, con quien las lanzo y que parametro se uso.

Que hace

- GET /api/historial devuelve la lista paginada del usuario actual, con filtro opcional por herramienta.
- GET /api/historial/:id devuelve el detalle y los archivos generados.
- DELETE /api/historial/:id borra una ejecucion del historial.
- Cada ejecucion conserva el objetivo, el estado, la duracion y los parametros ya enmascarados.
- Un usuario solo ve lo suyo; el historial no es compartido entre cuentas.

Para que sirve

Para no repetir la misma comprobacion cada vez que alguien pregunta, y para recuperar el informe de una ejecucion anterior.

Que soluciona

- Repetir un diagnostico que ya se habia hecho: si la respuesta no cambio, volver a lanzarlo wastes tiempo y cuota de terceros.
- Perder un informe que estaba en pantalla y no se guardo.
- La confusion de quien lanzo una ejecucion: cada registro lleva su autor.

Flujo

1. Tras una ejecucion, la entrada aparece sola en el historial reciente del menu lateral.
2. Se abre /historial para verlas todas, con filtro por herramienta.
3. Al abrir una, se ve el detalle completo y se puede volver a descargar en cualquier formato.
4. Se puede borrar la entrada si ya no interesa.

Informes y enlaces compartidos

Como se entrega el resultado y como se comparte con alguien de fuera.

Que hace

- El mismo resultado se renderiza en PDF, HTML, JSON, Markdown y TXT.
- El renderizado se hace siempre desde el resultado guardado, no desde un informe ya impreso, de modo que corregir el formato mejora todo el historico.
- Una ejecucion se puede compartir mediante un enlace publico temporal, con caducidad configurable entre 1 y 90 dias.
- El enlace usa un token opaco que no revela el identificador interno ni el autor.
- Quien tiene el enlace puede abrir el informe sin iniciar sesion, y el enlace deja de funcionar al expirar o al revocarlo.
- El contenido publicado pasa antes por el enmascarado de secretos.

Para que sirve

Para entregar a un cliente el resultado de un diagnostico sin darle acceso a la herramienta, y con la garantia de que el enlace dejara de servir dentro de un plazo.

Que soluciona

- Enviar un informe que caduca mal y queda accesible para siempre.
- Dar credenciales de la herramienta a alguien que solo necesitaba leer un resultado.
- Tener cinco versiones distintas del mismo informe segun el formato, que se contradicen entre si.

Flujo

1. Se lanza la comprobacion y, si procede, se marca la casilla de compartir y el numero de dias de validez.
2. El servidor crea el enlace y devuelve su URL y su fecha de caducidad.
3. Se copia el enlace y se envia a quien corresponda.
4. Quien lo abre ve el informe en HTML sin necesidad de cuenta.
5. Pasados los dias indicados, el enlace devuelve un aviso de caducidad; el propietario puede revocarlo antes.

Metricas

Contadores y tiempos de las peticiones, para vigilar el comportamiento del servicio.

Que hace

- Cuenta peticiones HTTP por metodo, ruta y codigo de respuesta.
- Mide la duracion de cada peticion.
- Sirve el volcado de las metricas en formato Prometheus, para integrarlas en un sistema de monitorizacion.
- No registra ni valores enviados por el usuario ni credenciales: solo metodo, ruta, estado y tiempo.

Para que sirve

Para saber si el servicio esta lento o saturado, y para detecting abnormalidades antes de que se conviertan en una caida.

Que soluciona

- Descubrir un problema de rendimiento cuando ya lo estan sufriendo los usuarios.
- Tener datos de monitoring de la aplicacion sin instrumentarla a mano.

Flujo

1. Cada peticion pasa por el middleware de metricas.
2. Al terminar la respuesta, se incrementa el contador y se observa la duracion.
3. El volcado se consulta para verlos en Grafana, Prometheus u otra herramienta compatible.

Produccion

Informe generado automaticamente por netlab

Estado de la revision

Hay 1 comprobacion(es) que impiden el despliegue y 1 aviso(s) que conviene resolver antes de exponer el servicio.

Bloqueantes

AUTH_ENABLED activa

FALLA

Con AUTH_ENABLED=false cualquiera que llegue al puerto puede lanzar diagnosticos. Es obligatorio en produccion.

Avisos

Cookie marcada Secure

REVISAR

El atributo Secure se activa solo si la peticion llega por HTTPS. Sin HTTPS, la cookie viaja sin ese atributo.

Correcto

Contrasenasy con hash scrypt	OK
Sesion en memoria, no en disco	OK
HOST no expuesto a Internet sin TLS	OK
SQLite en data/	OK
Primer administrador creado	OK
Compilacion de la web	OK
Directorio data/ escribible	OK
Claves de terceros presentes	OK
.env ignorado por git	OK
Ausencia de secretos en el historial	OK
Topes por minuto y por usuario	OK
Politica de seguridad de contenidos	OK

Resumen por area

Area	Comprobaciones	Estado
Autenticacion	2	Falla
Sesiones	2	Revisar
Red	1	Correcto
Base de datos	3	Correcto
Frontend	1	Correcto
Secretos	3	Correcto
Limites	1	Correcto
Cabeceras	1	Correcto

Herramientas

Informe generado automáticamente por netlab

Catalogo

7 herramientas disponibles

La lista no esta escrita en ningun sitio del servidor: se construye leyendo los ficheros de src/tools al arrancar. Anadir una comprobacion nueva es dejar el archivo, sin tocar el servidor ni la interfaz.

Comprobador de DNS

Identificador: dns-checker

Consulta los registros de un dominio y avisa de lo que está mal: SPF, DMARC, IPv6, alias o discrepancias entre resolvers.

Campos Dominio o nombre de host, Tipos de registro, Resolución inversa de una IP, Comparar con un archivo .txt, Nombre del archivo, Archivo, Comparar con Cloudflare y Google, Tiempo de espera por consulta (ms)

Requiere salida a internet Si

Reputacion de IP

Identificador: ip-abuse

Consulta AbuseIPDB para saber si una IP tiene avisos de abuso, de cuantas fuentes independientes y de que tipo.

Campos Direccion IP, Ventana de busqueda, Incluir los avisos uno a uno, Incluir los comentarios de quien reporta, Maximo de avisos a detallar, Tiempo de espera

Requiere salida a internet Si

Auditoria de IP

Identificador: ip-audit

Quien registro la IP, que nombre inverso tiene, si aparece en listas negras de correo y si figura como controlador de botnet. Ojo: BCL no se puede comprobar desde un servidor sin resolver; ver la seccion de botnet C&C.

Campos Direccion IP, Consultar listas negras, Cuantas listas consultar, Pedir el motivo de los listados, Verificar en las zonas de pago de Spamhaus, Tiempo de espera

Requiere salida a internet Si

Comprobador de correo

Identificador: mail-checker

Puntúa de 0 a 10 la configuración de correo de un dominio (MX, SPF, DKIM, DMARC, PTR, listas negras y transporte) o de un correo enviado (.eml), al estilo de mail-tester.

Campos Dominio, Analizar un correo enviado (.eml) en vez de un dominio, Archivo del correo (.eml), O pega aquí el código fuente, Selectores DKIM, Comprobar MTA-STX, TLS-RPT y BIMI, Comprobar listas negras, Tiempo de espera por consulta

Requiere salida a internet Si

Validador SMTP

Identificador: smtp-validator

Prueba un servidor SMTP con los datos que escribas: conexión, STARTTLS o TLS, capacidades, autenticación y, si lo pides, un envío real y una sesión IMAP.

Campos	Servidor SMTP, Puerto, Cómo empieza la conexión, Usuario, Contraseña, Exigir que el certificado sea válido y esté emitido por una autoridad de confianza, Espera máxima por paso, Enviar además un correo de prueba, Remitente del correo de prueba, Destinatario del correo de prueba, Asunto del correo de prueba, Probar también IMAP con las mismas credenciales, Servidor IMAP, Puerto IMAP
Requiere salida a internet	Si

Analizador de subredes

Identificador: subnet-analyzer

Calcula el direccionamiento de una red, reparte subredes iguales o hace un reparto VLSM por tamaño de cada tramo.

Campos	Red, Reparto en subredes iguales, Reparto VLSM por tamaño, Incluir el listado de direcciones, Máximo de direcciones a listar
Requiere salida a internet	Si

Comprobador de sitios web

Identificador: web-checker

Comprueba si un sitio responde y, si algo falla, dice por qué: DNS, TLS, redirecciones, caducidad del dominio y si hay una página de venta delante.

Campos	Dirección del sitio, Mostrar opciones avanzadas, Tiempo de espera (ms), Permitir direcciones internas, Consultar la caducidad del dominio (RDAP), Comparar con Cloudflare y Google, Crear enlace público temporal, Días de validez del enlace
Requiere salida a internet	Si

Despliegue

Informe generado automaticamente por netlab

Requisitos

- Node.js 24 o superior. La version importa: la base de datos usa el modulo de SQLite integrado.
- 512 MB de memoria como minimo; 1 GB recomendado.
- Un proxy inverso con HTTPS delante, para no exponer la contraseña en claro.

Orden de despliegue

1. Instalar Node.js 24 en el servidor.
2. Copiar el proyecto, instalar dependencias con npm ci y compilar la interfaz con npm run build.
3. Crear el archivo .env a partir de .env.example con AUTH_ENABLED=true y protegerlo con chmod 600.
4. Dejar el servidor escuchando solo en localhost, con HOST=127.0.0.1, y publicarlo con el proxy inverso.
5. Dar permisos de escritura al usuario del servicio sobre el directorio data.
6. Arrancar el servicio y comprobar el estado.
7. Abrir la pagina: aparecera el formulario del administrador principal. Crear la cuenta y entrar.
8. Las cuentas del administrador principal y del resto de usuarios se dan de alta desde la propia aplicacion. AUTH_USER y AUTH_PASSWORD_HASH del .env ya no se usan: se pueden borrar.
9. Configurar el proxy con HTTPS y verificar que la cookie de sesion llega marcada como Secure.

Verificacion tras el despliegue

- La pagina responde y muestra el formulario de administrador principal.
- El login funciona con la cuenta creada.
- Una ejecucion de prueba se completa y aparece en el historial.
- El informe se descarga en PDF.
- GET /api/sesion responde autenticado con el rol esperado.
- El proxy responde con el certificado valido y la peticion llega por HTTPS.

Copia de seguridad

Lo unico que hay que respaldar es el directorio data: usuarios, historial, enlaces compartidos y auditoria. El .env tambien, porque contiene las credenciales de los servicios externos, pero no debe ir al repositorio.